# 32.2 DM 配对系统

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~260k tokens，输出 ~7k tokens（本节）

DM 配对（Pairing）是 OpenClaw 控制谁可以与 Bot 私聊的核心安全机制。它实现了一个类似 Bluetooth 设备配对的流程：陌生用户首次发消息时获得一个配对码，Bot 所有者在终端审批后该用户才被加入白名单。本节从存储层到 CLI 完整剖析这套系统。

## 32.2.1 配对流程：配对码生成 → 用户审批 → 白名单持久化

### 核心数据结构

配对系统涉及两个 JSON 文件，存储在 `~/.openclaw/oauth/` 目录下：

| 文件 | 内容 | 格式 |
|---|---|---|
| `{channel}-pairing.json` | 待审批的配对请求 | `{ version: 1, requests: PairingRequest[] }` |
| `{channel}-allowFrom.json` | 已审批的白名单 | `{ version: 1, allowFrom: string[] }` |

`PairingRequest` 的结构如下：

```
// src/pairing/pairing-store.ts

type PairingRequest = {
  id: string;           // 用户标识（如电话号码、Telegram ID）
  code: string;         // 8 位人类友好的配对码
  createdAt: string;    // ISO 8601 创建时间
  lastSeenAt: string;   // 最近一次尝试的时间
  meta?: Record<string, string>;  // 额外元数据（如用户名）
};
```

### 配对码设计

配对码是 8 位大写字母和数字的组合，刻意排除了容易混淆的字符：

```ts
// src/pairing/pairing-store.ts

const PAIRING_CODE_LENGTH = 8;
const PAIRING_CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
// 注意排除了：0（与 O 混淆）、1（与 I 混淆）、I、O

function randomCode(): string {
  let out = "";
  for (let i = 0; i < PAIRING_CODE_LENGTH; i++) {
    const idx = crypto.randomInt(0, PAIRING_CODE_ALPHABET.length);
    out += PAIRING_CODE_ALPHABET[idx];
  }
  return out;
}
```

> **衍生解释**：排除易混淆字符的做法在人类可读编码中很常见。著名的例子包括 Base32 编码（RFC 4648）排除了 0、1、8，以及 Crockford 的 Base32 变体。这里使用 `crypto.randomInt` 而非 `Math.random`，因为后者是伪随机数生成器（PRNG），在安全场景中不应使用——`crypto.randomInt` 使用操作系统的密码学安全随机源（如 Linux 的 `/dev/urandom`）。

## 完整配对流程

```
┌──────────┐   DM 消息   ┌──────────┐   创建/更新请求   ┌──────────────────┐
│ 未知用户 │──────────→│ 渠道插件 │──────────────→│ pairing-store.json │
└──────────┘             └──────────┘                   └──────────────────┘
                              │                                  │
                              │  返回配对消息                    │ 生成 8 位配对码
                              ↓                                  │
                          ┌──────────┐                           │
                          │ 用户收到 │←──────────────────────────┘
                          │ 配对码   │
                          └──────────┘

                  ┌──────────┐   approve <code>   ┌──────────────────┐
                  │ Bot 所有者│──────────────────→│ approveChannelPairing│
                  │ (CLI)    │                    │ Code              │
                  └──────────┘                    └──────────────────┘
                                                        │
                                                   ┌────┴────┐
                                                   │ 1. 查找匹配码  │
                                                   │ 2. 从 requests │
                                                   │    中移除      │
                                                   │ 3. 加入        │
                                                   │    allowFrom   │
                                                   └────────────────┘
```

## 配对请求管理

`upsertChannelPairingRequest` 创建或更新配对请求，同时管理过期和容量限制：

```
// src/pairing/pairing-store.ts（简化）

const PAIRING_PENDING_TTL_MS = 60 * 60 * 1000;  // 1 小时过期
const PAIRING_PENDING_MAX = 3;                    // 最多 3 个并发请求

export async function upsertChannelPairingRequest(params) {
  return await withFileLock(filePath, fallback, async () => {
    // 1. 读取现有请求
    const { value } = await readJsonFile(filePath, fallback);
    let reqs = value.requests;

    // 2. 清理过期请求
    const { requests: pruned } = pruneExpiredRequests(reqs, Date.now());
    reqs = pruned;

    // 3. 检查是否已存在（同一用户重复发消息）
    const existingIdx = reqs.findIndex((r) => r.id === id);
    if (existingIdx >= 0) {
      // 更新 lastSeenAt，保留原有配对码
      reqs[existingIdx] = { ...existing, lastSeenAt: now };
      return { code: existing.code, created: false };
    }

    // 4. 容量检查（最多 3 个）
    if (reqs.length >= PAIRING_PENDING_MAX) {
      return { code: "", created: false };  // 拒绝新请求
    }

    // 5. 生成唯一配对码并创建新请求
    const code = generateUniqueCode(existingCodes);
    reqs.push({ id, code, createdAt: now, lastSeenAt: now, meta });
    await writeJsonFile(filePath, { version: 1, requests: reqs });
    return { code, created: true };
  });
}
```

安全设计要点：

- **1 小时过期**：防止配对码被长时间暴露
- **最多 3 个并发请求**：防止攻击者通过大量发送消息生成海量配对请求（拒绝服务攻击）
- **唯一码保证**： `generateUniqueCode` 最多尝试 500 次生成不重复的码
- **文件锁**： `withFileLock` （基于 `proper-lockfile` 库）确保并发访问安全

## 审批与白名单

`approveChannelPairingCode` 完成配对的最后一步——验证配对码、从请求列表移除、加入白名单：

```
// src/pairing/pairing-store.ts（简化）

export async function approveChannelPairingCode(params) {
  const code = params.code.trim().toUpperCase();

  return await withFileLock(filePath, fallback, async () => {
    const reqs = /* 读取并清理过期请求 */;
    const idx = reqs.findIndex((r) => r.code.toUpperCase() === code);
    if (idx < 0) return null;  // 未找到匹配码

    const entry = reqs[idx];
    reqs.splice(idx, 1);  // 从请求列表移除
    await writeJsonFile(filePath, { version: 1, requests: reqs });

    // 加入白名单
    await addChannelAllowFromStoreEntry({
      channel: params.channel,
      entry: entry.id,
    });

    return { id: entry.id, entry };
  });
}
```

## 原子写入保护

所有文件写入都使用原子写入模式（与配置系统相同）：

```
async function writeJsonFile(filePath: string, value: unknown): Promise<void> {
  const dir = path.dirname(filePath);
  await fs.promises.mkdir(dir, { recursive: true, mode: 0o700 });   // 目录权限 700
  const tmp = path.join(dir, `${path.basename(filePath)}.${crypto.randomUUID()}.tmp`);
  await fs.promises.writeFile(tmp, JSON.stringify(value, null, 2) + "\n");
  await fs.promises.chmod(tmp, 0o600);                              // 文件权限 600
  await fs.promises.rename(tmp, filePath);                          // 原子重命名
}
```

## 32.2.2 配对 CLI（`src/cli/pairing-cli.ts`）

### 命令结构

配对 CLI 注册在 `openclaw pairing` 子命令下，提供两个操作：

```
openclaw pairing list [channel]        # 列出待审批的配对请求
openclaw pairing approve <channel> <code>  # 审批配对码
```

### list 命令

列出指定渠道的待审批请求，支持表格和 JSON 两种输出格式：

```
// src/cli/pairing-cli.ts（简化）

pairing.command("list")
  .argument("[channel]", "Channel (whatsapp, telegram, ...)")
  .option("--json", "Print JSON")
  .action(async (channelArg, opts) => {
    const channel = parseChannel(channelRaw, channels);
    const requests = await listChannelPairingRequests(channel);

    if (opts.json) {
      log(JSON.stringify({ channel, requests }, null, 2));
      return;
    }
    if (requests.length === 0) {
      log("No pending pairing requests.");
      return;
    }
    // 表格输出：Code | ID | Meta | Requested
    log(renderTable({
      columns: [
        { key: "Code", header: "Code" },
        { key: "ID", header: resolvePairingIdLabel(channel) },
        { key: "Meta", header: "Meta" },
        { key: "Requested", header: "Requested" },
      ],
      rows: requests.map(/* ... */),
    }));
  });
```

`resolvePairingIdLabel` 根据渠道类型返回友好的 ID 标签——WhatsApp 显示"Phone"，Telegram 显示"User ID"等。

## approve 命令

审批并可选地通知请求者：

```
pairing.command("approve")
  .argument("<codeOrChannel>")
  .argument("[code]")
  .option("--notify", "Notify the requester")
  .action(async (codeOrChannel, code, opts) => {
    const channel = parseChannel(channelRaw, channels);
    const approved = await approveChannelPairingCode({ channel, code });
    if (!approved) {
      throw new Error("No pending pairing request found for code");
    }
    log(`Approved ${channel} sender ${approved.id}.`);

    if (opts.notify) {
      await notifyPairingApproved({ channelId: channel, id: approved.id, cfg });
    }
  });
```

`--notify` 选项通过渠道插件的 `pairing.notifyApproval` 接口，向刚被审批的用户发送通知消息。

## 渠道适配器

每个渠道插件可以通过 `ChannelPairingAdapter` 接口定制配对行为：

```
// src/channels/plugins/types.ts（概念）

type ChannelPairingAdapter = {
  idLabel: string;                                          // ID 的人类可读标签
  normalizeAllowEntry?: (entry: string) => string;         // 标准化白名单条目
  notifyApproval?: (params: { cfg; id; runtime }) => void; // 审批后通知
};
```

例如 WhatsApp 的适配器可能将电话号码标准化为国际格式（如去除 `+` 前缀和空格），确保白名单匹配的一致性。

## 32.2.3 DM 策略： `pairing` / `open`

### 策略枚举

每个渠道的 DM 接收策略通过 `dmPolicy` 配置：

| 策略 | 行为 |
| --- | --- |
| `"pairing"` | 默认策略。未知用户收到配对码，需要 Bot 所有者审批 |
| `"open"` | 任何人都可以发 DM 并得到响应（需配合 `allowFrom: ["*"]` ） |
| `"disabled"` | 完全禁用 DM 功能 |

### open 策略的安全检查

安全审计系统（24.4 节）会对 `open` 策略发出严重警告：

```
// src/security/audit.ts（简化）

if (dmPolicy === "open") {
  findings.push({
    checkId: `channels.${provider}.dm.open`,
    severity: "critical",                    // 最高严重级别
    title: `${label} DMs are open`,
    detail: `dmPolicy="open" allows anyone to DM the bot.`,
    remediation: "Use pairing/allowlist...",
  });
}
```

此外，如果 `dmPolicy` 设为 `open` 但 `allowFrom` 中未包含通配符 `"*"` ，审计系统还会报告配置不一致：

```
channels.whatsapp.dm.open_invalid: "open" requires allowFrom to include "*".
```

### 会话隔离与多用户 DM

当多个用户可以 DM（ `open` 策略或白名单中有多人）， `session.dmScope` 配置决定了会话隔离级别：

| dmScope | 行为 |
| --- | --- |
| `"main"` | 所有 DM 发送者共享主会话（**存在上下文泄漏风险**） |
| `"per-channel-peer"` | 每个渠道+发送者对应一个独立会话 |

| dmScope | 行为 |
| --- | --- |
| `"per-account-channel-peer"` | 每个账号+渠道+发送者对应独立会话（多账号渠道） |

安全审计在检测到 `dmScope="main"` 且存在多个 DM 发送者时，会发出警告：

```
if (dmScope === "main" && isMultiUserDm) {
  findings.push({
    severity: "warn",
    title: `${label} DMs share the main session`,
    detail: "Multiple DM senders share the main session, which can leak context across users
    remediation: 'Set session.dmScope="per-channel-peer" to isolate DM sessions per sender.'
  });
}
```

# 本节小结

1. **配对码**：8 位字母数字组合，排除易混淆字符（0、O、1、I），使用密码学安全随机生成。
2. **三步流程**：未知用户发 DM → 收到配对码 → Bot 所有者 CLI 审批 → 加入白名单。
3. **安全限制**：请求 1 小时过期，最多 3 个并发待审批请求，文件锁防并发。
4. **原子写入**：tmp 文件 + rename 模式，文件权限 0o600，目录权限 0o700。
5. **渠道适配器**：每个渠道可以自定义 ID 标签、白名单标准化规则和审批通知方式。
6. **DM 策略三级**：`pairing`（默认安全）、`open`（显式放开，审计告警）、`disabled`（完全禁用）。
7. **会话隔离**：`dmScope` 配置防止多用户 DM 共享上下文导致的信息泄漏。